# Unity Notes

Ario Bashiri
ariobashiri.com

# Physics

**Rigidbody**

- Full physics simulation — gravity, forces, momentum, collisions
- Best for: realistic movement, rolling balls, pucks, ragdolls

Rigidbody Settings Cheat Sheet

| Setting | What it does |
|---|---|
| Mass | Weight, affects force reactions |
| Linear Damping | Slows down movement over time |
| Angular Damping | Slows down spinning over time |
| Is Kinematic | Ignores physics forces, moved only by code/transform |
| Collision Detection | Discrete (default) vs Continuous/Continuous Dynamic (fast objects) |
| Freeze Position/Rotation | Locks specific axes |

**Character Controller**

- No physics, manual movement control
- Best for: precise player movement (FPS, platformers)

**Input (old system)**

- UnityEngine.Input — legacy way of reading input, used inside physics/movement scripts

# Input Management

**Input Manager (absolute banger)**

- The new Input System package
- Uses .inputactions asset, Action Maps, event-driven callbacks (OnMove, OnJump, etc.)
- Supports rebinding, multiple devices, better overall

**Input (old one)**

- UnityEngine.Input.GetKey(), Input.GetAxis()
- Polling-based, simple, but no built-in rebinding or multi-device support

# Camera Follow

**Code**

- Bad — manual transform.position / Quaternion math to follow the player
- Full control but painful to get right (tunneling, distortion, jitter — all the issues we debugged this whole conversation)

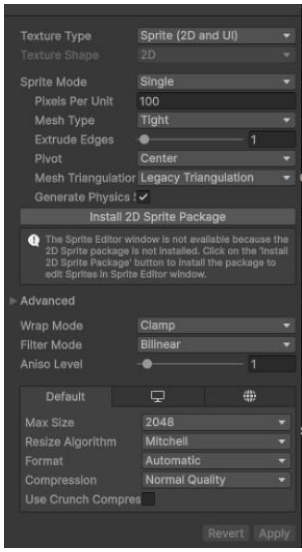
**Camera as child object on Player**

- Simplest method — just parent the camera in Hierarchy
- Automatically follows position and rotation
- Downside: less flexible for camera switching/transitions

**Cinemachine (The best imo) (installed through Package Manager)**

- Unity's dedicated camera system package
- Handles follow, look-at, collision avoidance, smooth transitions, shake, and more — all without custom code
- Industry standard for camera work in Unity, used in most professional projects

# UI




A canvas must be the parent holding UI elements.
Canvas Scalar better be: **“Scale With Screen Size”** + **“Match Width Or Height”**

- For text: **TextMeshPro** is much better with many more options.
- For images: **Panel** (set sprite image onto it) (**“Set Native Size”** is really useful)

# Useful Scripts

- FindAnyObjectByType<T>() : Finds an object automatically without dragging into Inspector
- GetComponent<T>() : Finding and getting objects/components (on the same object)
- GetComponentInChildren<T>()
- GetComponentInParent<T>()
- Destroy(gameObject)
- Destroy(gameObject, 3f) : destroy after delay
- gameObject.SetActive(false) : disables/enables the object
- gameObject.CompareTag("Enemy")
- gameObject.layer = LayerMask.NameToLayer("Ground")

# Prefabs are useful. *“Prefab everything that you can prefab”*

Prefab in other words is saving an object. It is better for **duplicating**, **better managing** objects, applying **group changes** to objects, etc.

# Empty Parent Object trick

Put the model you want to attach code to as a child of an empty object. Changing the model after some time won’t cause any loss since all the data are on the empty parent object.

**NOTE**: be sure to put the **parent object’s position** at the **center** (**center of mass**) of the **child object**. That will fix any rotational bug.

# Unity Order of Execution

- Awake() → runs once, when object is first created
- OnEnable() → runs every time the object becomes active
- Start() → runs once, after Awake, before first Update
- Update() → runs every frame

# RAYCAST IS THE MOST AMAZING THING I'VE EVER SEEN

It shoots an **invisible line** from a point in a direction and tells you what it hits first.
Used for:

- Player "look at object to interact" (E to pick up, open doors, etc.)
- Shooting/hit detection
- Ground cheks
- AI sight
- Etc.

How to use it for object interaction:
You must set the **Layer** to "**Interactable**" on the target objects. And other Layers for other use cases.
For instance, for AI bots to see player, the raycast must check if the collision has occurred to an item with the **Layer** being "**Player**"

```
void Start()
{
    // Get the Camera component from the PlayerLook script (so raycast starts from where player is looking)
    cam = GetComponent<PlayerLook>().cam;

    // Get reference to the UI script that displays interaction prompts
    playerUI = GetComponent<PlayerUI>();

    // Get reference to the Input Manager script that reads player input
    inputManager = GetComponent<InputManager>();
}

void Update()
{
    // Clear the prompt text every frame first (resets if nothing is being looked at)
    playerUI.UpdateText(string.Empty);

    // Create a ray starting from the camera's position, going forward in the direction the camera is facing
    Ray ray = new Ray(cam.transform.position, cam.transform.forward);

    // Draw the ray visually in the Scene view for debugging (only visible in Editor, not in builds)
    Debug.DrawRay(ray.origin, ray.direction * distance);

    // Variable to store information about whatever the raycast hits
    RaycastHit hitInfo;

    // Fire the raycast. "out hitInfo" means the function will fill hitInfo with collision data if it hits something.
    // "distance" limits how far the ray checks, "mask" limits which layers it can detect (ignores irrelevant objects)
    if (Physics.Raycast(ray, out hitInfo, distance, mask))
    {
        // Check if the object we hit has an Interactable component attached
        if (hitInfo.collider.GetComponent<Interactable>() != null)
        {
            // Store a reference to that Interactable component
            Interactable interactable = hitInfo.collider.GetComponent<Interactable>();

            // Show that object's custom prompt message (e.g. "Press E to open door")
            playerUI.UpdateText(interactable.promptMessage);

            // Check if the Interact input action was pressed this frame
            if (inputManager.onFoot.Interact.triggered)
            {
                // Call the interact function on that object (opens door, picks up item, etc.)
                interactable.BaseInteract();
            }
        }
    }
}
```

```
public bool CanSeePlayer()
{
    if(player != null)
    {
        // is the player close enough to be seen?
        if (Vector3.Distance(transform.position, player.transform.position) < sightDistance)
        {
            Vector3 targetDirection = player.transform.position - transform.position - (Vector3.up * eyeHeight);
            float angleToPlayer = Vector3.Angle(targetDirection, transform.forward);
            // is the player within the field of view of enemy?
            if (angleToPlayer >= -fieldOfView && angleToPlayer <= fieldOfView)
            {
                Ray ray = new Ray(transform.position + (Vector3.up * eyeHeight), targetDirection);
                Debug.DrawRay(ray.origin, ray.direction * sightDistance);

                RaycastHit hitInfo = new RaycastHit();

                // is enemy's sight blocked by any object?
                if (Physics.Raycast(ray, out hitInfo, sightDistance))
                {
                    if (hitInfo.transform.gameObject == player)
                    {
                        return true;
                    }
                }

            }
        }
    }
    return false;
}
```

← Enemy detection using Raycast
sightDistance = 20f
fieldOfView = 85f

## FixedUpdate() vs Update()

- **Update()** is recommended to be used for **game logic**. Must use **Time.DeltaTime** for this.
- **FixedUpdate()** is recommended to be used for **Physics**. Must use **Time.FixedDeltaTime** for this.

## AddForce() vs AddRelativeForce()

- **AddForce()** adds a **global** force (relateive to the **world's x, y or z** axis)
- **AddRelativeForce()** adds a **Local** force (relative to **the object's x, y and z** axes)